

1. Visão geral dos arquivos

Arquivo	Responsabilidade	Páginas que carregam
js/core.js	Base: armazenamento, movimentações, estatísticas e preferências do usuário	Todas
js/dashboard.js	Dashboard: cards, lista, gráficos, modal de transações, metas e cards 3D	index.html
js/transacoes.js	Página de transações: lista com filtros e edição	transacoes.html
js/relatorios.js	Relatórios: médias, maior gasto e gráfico anual	relatorios.html
js/configuracoes.js	Configurações: nome, tema e reset de dados	configuracoes.html
js/gamification.js	Gamificação: XP, níveis, conquistas, metas e score de saúde	index.html
js/background3d.js	Fundo 3D animado — exclusivamente visual , fora do escopo	Todas

Ordem de carga no index.html: core.js → (stats.js/user.js referenciados mas não existentes) → gamification.js → background3d.js → dashboard.js.

2. core.js — camada de dados

2.1 Objeto finance (movimentações)

Função	Entradas	Comportamento	Retorno
salvar()	—	Persiste finance.items no localStorage	void
carregar()	—	Lê movimentações do localStorage	void
listar()	—	Acesso direto ao array de itens	Array<item>
adicionar({tipo, descricao, valor, categoria, data})	objeto	Cria item com id = Date.now(), adiciona ao array e salva	item criado
editar(id, dados)	id, dados	Mescla os campos no item correspondente e salva	boolean (editou)
excluir(id)	id	Remove o item e salva	void
getById(id)	id	Localiza item por id	item undefined
filtrarPorTipo(tipo)	"entrada" "saida"	Filtra por tipo	Array
filtrarPorCategoria(categoria)	string	Filtra por categoria exata	Array
filtrarPorPeriodo(inicio, fim)	datas	Filtra entre duas datas	Array
buscar(termo)	string	Busca por descrição (case-insensitive)	Array
totalEntradas()	—	Soma valores do tipo "entrada"	number
totalSaidas()	—	Soma valores do tipo "saida"	number
saldo()	—	totalEntradas() – totalSaidas()	number
totalAPagar()	—	Alias de totalSaidas()	number
resetarTudo()	—	Limpa o localStorage e zera itens	void

2.2 Objeto storage

Função	Comportamento
salvarMovimentacoes(itens)	Grava na chave <code>finance.movimentacoes</code>

carregarMovimentacoes()	Lê e retorna array; default []
salvarConfig(cfg)	Grava na chave <code>finance.config</code>
carregarConfig()	Retorna config; default {nome:"", tema:"escuro"}
limparTudo()	<code>localStorage.clear()</code>

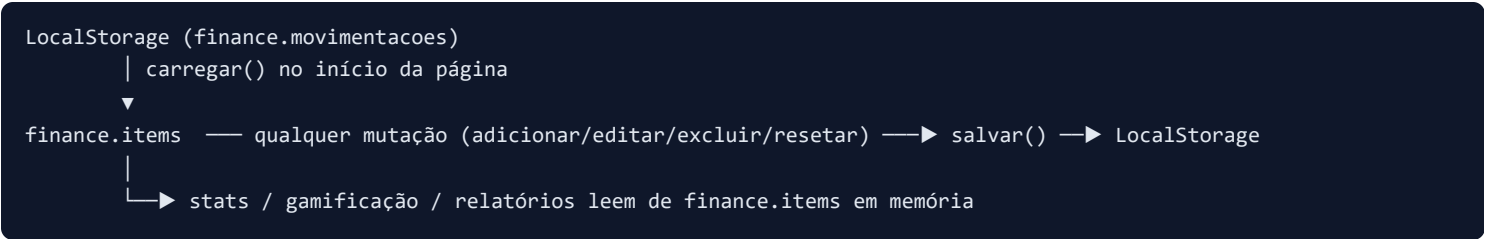
2.3 Objeto stats (mesclado a finance via Object.assign)

Função	Comportamento	Retorno
porCategoria(tipo, items?)	Agrupar valores por categoria	{categoria: total}
resumoMensal(mes, ano, items?)	Filtrar pelo mês/ano da data do item	{entradas, saidas, saldo}
maiorGasto(items?)	Maior saída	item null
mediaMensal(items?)	Total de saídas ÷ meses com dados	number

2.4 Objeto userPrefs (mesclado a finance via Object.assign)

Função	Comportamento
getUsuario()	Nome salvo na config ("" se vazio)
setUsuario(nome)	Persiste nome e re-renderiza perfil; dispara avaliarConquistas()
getTema()	"claro" "escuro"
setTema(tema)	Persiste e aplica o tema
aplicarTema(tema)	Define CSS vars (--bg, --text, etc.) e a classe theme-light no body
atualizarPerfilUI()	Atualiza avatar (inicial), saudação e badge de nível do cabeçalho

2.5 Fluxo de persistência



Regra geral: ao carregar uma página, chama-se `finance.carregar()` primeiro; toda alteração grava imediatamente, e toda tela faz render a partir da memória.

3. Dashboard (js/dashboard.js)

3.1 Funções principais

Função	Comportamento
obterIconeCategoria(item)	Retorna emoji por tipo/categoria/descrição (entrada → 🍷 ; categorias mapeadas; default 🍀)
atualizarTela()	Central do dashboard: calcula saldo/receitas/despesas/economia, preenche cards e subtextos, renderiza gamificação (renderUI + avaliarConquistas), monta a lista das 6 últimas movimentações e chama atualizarGraficos()
deletarItem(id)	finance.excluir(id) + atualizarTela()
atualizarGraficos()	Recria (destroy + new) o gráfico de pizza por categoria e o de linha dos últimos 8 meses (Entradas, Despesas e Saldo), com legenda
setModalMode(editing)	Altera título do modal e texto do botão ("Nova transação" ↔ "Editar transação")
openModal()	Zera editingId, modo "nova" e abre o modal
closeModal()	Fecha o modal
editarItem(id)	Preenche o formulário com finance.getById(id) e abre em modo edição (editingId = id)

3.2 Fluxo: adicionar / editar transação (submit do #transactionForm)

```
1. Validação prevent default
2. editingId === null?
   | SIM (novo): calcula data de hoje (fuso local) → finance.adicionar({tipo, descricao, valor, categoria, data})
   |               → gamification.adicionarXp(10) ; se for ENTRADA → dispararConfetti()
   | NÃO (edição): finance.editar(editingId, dados) → gamification.adicionarXp(3)
3. editingId = null ; form.reset() ; closeModal() ; atualizarTela()
```

3.3 Fluxo: metas (submit do #goalForm)

```
1. Lê descricao, tipo e valor
2. Guarda: exige descricao e valor > 0 e gamification definido
3. gamification.adicionarMeta({descricao, tipo, meta:valor}) ; form.reset()
```

3.4 Fluxo: exclusão de transação

```
1. botão 🗑️ → deletarItem(id)
2. finance.excluir(id) → atualizarTela() (reavalia gamificação e gráficos)
```

Nota: os efeitos de cards 3D e confete são visuais (fora do escopo deste documento). O confete é disparado apenas como reforço de feedback em entradas.

4. Página de Transações (js/transacoes.js)

4.1 Funções

Função	Comportamento
renderList()	Lê finance.listar() invertido (mais recentes primeiro), aplica filtros (busca por descrição, tipo, categoria), monta os cards e os botões ✏️ / 🗑️
deletarItem(id)	finance.excluir(id) + renderList()
setModalMode / openModal / closeModal / editarItem	Mesmos conceitos do dashboard (modal de edição)

4.2 Fluxo de filtros

```
Eventos: input(#filterDesc) + change(#filterTipo) + change(#filterCat) —▶ renderList()
renderList:
  1. itens = finance.listar().reverse()
  2. se há descrição → finance.buscar(desc).reverse()
  3. se há tipo → filtra i.tipo === tipo
  4. se há categoria → filtra i.categoria === categoria
  5. renderiza lista ou mensagem "Nenhuma transação encontrada"
```

4.3 Fluxo de edição

```
1. ✏️ → editarItem(id): preenche form (descricao, valor, tipo, categoria) e abre modal em modo edição
2. Submit → finance.editar(id, dados) → renderList()
   (edição nesta página NÃO concede gamificação – o gamification só está no dashboard)
```

5. Relatórios (js/relatorios.js)

Item	Fluxo
Média mensal	finance.mediaMensal() → formatado em R\$ → #mediaVal
Maior gasto	finance.maiorGasto() → valor e descrição com data (dd/mm/aaaa) nos cards
Gráfico anual	Loop dos últimos 12 meses: finance.resumoMensal() → datasets Receitas (#19d3ff) e Despesas (#ff5c7c) em barras

Relatórios só leem dados — nenhuma mutação de estado acontece na página.

6. Configurações (js/configuracoes.js)

Função	Comportamento
loadData()	Preenche #configNome com o nome salvo e #configTema com o tema vigente
form.submit	finance.setUsuario(nome) + finance.setTema(tema) → atualiza avatar → mostra mensagem "salvo" por 3s
resetarDados()	Confirm() → storage.limparTudo() (ou localStorage.clear()) → reload da página

resetarDados() apaga TUDO do localStorage: movimentações, config e estado de gamificação.

7. Gamificação (js/gamification.js)

7.1 Estado persistido (chave finance.gamification)

```
{ xp: 0, conquistas: [], metas: [], ultimaVisita: null, metaConcluida: false, xpPagos: [] }
```

7.2 Funções

Função	Comportamento
carregar() / salvar()	Lê/grava o estado em localStorage (com defaults corretivos por campo)
init()	Carrega estado, cria container de toasts, concede +3 XP de visita diária (1×/dia), renderUI → avaliarConquistas → renderUI
xpParaNivel(nivel)	Curva crescente: $220 + (\text{nivel} - 1) \times 70$ XP para subir do nível N ao N+1
getLevelInfo()	Converte XP total em nível (teto 50), XP do nível atual, % de progresso, título/ícone vigente e próximo nível
adicionarXp(qtd, motivo)	Soma XP, salva, e se subiu de nível notifica (cor ouro); re-renderiza
_melhorEconomiaMensal()	Maior razão (entradas-saídas)/entradas entre todos os meses com entrada
avaliarConquistas()	Cascata: 1) revoga conquistas de estado que não valem mais; 2) desbloqueia novas (+15 XP na 1ª vez, via xpPagos); repete até estabilizar
calcularScoreSaude()	Score 0-100 com 6 componentes ponderados × fator de maturidade $\min(1, \text{total}/6)$
_animarScore(el, alvo)	Anima o número do valor atual até o alvo em 1,1s (easing cúbico)
listarMetas() / adicionarMeta({descricao,tipo,meta}) / excluirMeta(id)	CRUD de metas (chama renderUI + avaliarConquistas após mutações)
calcularProgressoMeta(meta)	Calcula progresso real pelo mês atual: $\text{economia} = \text{saldo}$, $\text{limite} = 1 - (\text{gasto}/\text{limite})$, $\text{receitas} = \text{entradas}$, $\text{transacoes} = \text{contagem}$
renderMetas()	Marca meta concluída (+15 XP, histórico metaConcluida) quando progresso ≥ 1; REABRE quando progresso < 1; salva mudanças
renderBadges()	Renderiza a grade de conquistas (destravada/travada)
renderUI()	Atualiza: ícone/título do nível, barra de XP, texto %, score (animado) e sincroniza badge do cabeçalho (atualizarPerfilUI)
dispararConfetti()	Feedback visual de reforço (fora do escopo)
notificar({icon,titulo,descricao,cor})	Cria toast no container #gamiToasts, some após 4,5s

7.3 Curva de níveis

Nível	XP para subir	Título	Ícone
1	220	Iniciante	
2	290	Aprendiz	
3	360	Explorador	
5	—	Estrategista	
8	—	Investidor	
12	—	Analista	
16	—	Especialista	

20	—	Mestre das Finanças	🔥
30	—	Lenda Financeira	🏆
40	—	Magnata	💎
50	—	Estilo de Vida	🚀

7.4 Fontes de XP (valores atuais)

Ação	XP	Observação
Cadastrar transação	+10	Dashboard; entrada ainda dispara confete
Editar transação	+3	Dashboard
Conquista desbloqueada	+15	Uma única vez por conquista (xpPagos)
Meta concluída	+15	Na 1ª vez que o requisito é atingido
Visita diária	+3	1× por dia (ultimaVisita)

7.5 Conquistas

ID	Título	Condição (teste)	Permanente
primeiro_passo	Primeiro passo	≥ 1 movimentação	sim
primeira_receita	Primeira receita	existe entrada	sim
movimentado5 / 25 / 50	Movimentado / Ritmo forte / Máquina	≥ 5 / 25 / 50 movimentações	sim
dia_movimentado	Dia movimentado	3 movimentações no mesmo dia	sim
saldo_5k ... saldo_100k	Saldo R\$ 5k a 100k	saldo ≥ 5000/10k/20k/30k/50k/70k/80k/100k	não (revogável)
economia_trinta / cinquenta	Economizador / Disciplina total	melhor economia mensal ≥ 30% / 50%	não
mes_positivo	Mês no positivo	saldo do mês atual > 0	não
meta_atingida	Meta atingida	histórico metaConcluida == true	sim
coleccionador3 / 6	Colecionador / Mestre colecionador	≥ 3 / 6 conquistas desbloqueadas	sim

7.6 Score de saúde financeira (0–100)

Componente	Fórmula	Max
Atividade	total de movimentações × 2	16
Receitas (histórico)	6 por mês com entrada (até 2 meses)	12
Saldo sólido	saldo ÷ total de entradas	14
Mês equilibrado	1 – (saídas do mês ÷ entradas do mês)	14
Disciplina histórica	1 – (saídas ÷ entradas)	8
Metas concluídas	3 por meta completa	10

Resultado final: soma × min(1, total/6) (fator de maturidade — sem histórico alto, o score não decola).

7.7 Fluxo de avaliação (avaliarConquistas)

```
Início
└─ [cascata] para cada iteração:
    1. REVOGA conquistas NÃO permanentes cuja condição falhou
```

```
2. DESBLOQUEIA novas conquistas (condição verdadeira)
  → +15 XP se ainda não paga (xpPagos)
3. se não houver novidade → para
↳ se mudou algo → salvar + renderUI
```

7.8 Fluxo de conclusão/rea abertura de metas (renderMetas)

```
para cada meta:
  progresso = calcularProgressoMeta(meta)
  └ progresso ≥ 1 e não concluída → conclui: +15 XP, metaConcluida=true, toast
  └ progresso < 1 e estava concluída → REABRE (completado=false)
    └ concluída e marca histórica não registrada → registra metaConcluida
salva se houve qualquer mudança ; renderiza cards ; se ganhou XP re-renderiza badges
```

8. Fluxos globais

8.1 Inicialização da aplicação (index.html)

1. Scripts síncronos (core, gamification, background3d, dashboard) parseiam
2. O parse do dashboard JÁ chama: `finance.carregar()` e `atualizarTela()`
3. `DOMContentLoaded`:
 - a. (core) `aplicarTema + atualizarPerfilUI` → badge de nível inicial
 - b. (gamification) `init()` → carrega estado, XP diário, `renderUI` + `avaliarConquistas`
 - c. (dashboard) `atualizarTela()` (2ª execução) → tela final idem
4. fim → interface interativa

8.2 Ciclo XP → nível → conquista

Ação de usuário (adicionar/editar/meta/visita)

- `adicionarXp(qtd)` → `estado.xp += qtd ; salvar()`
- `getLevelInfo()` → calcula nível via curva 220/70
- se subiu de nível → notificação "Nível subiu!"
- `renderUI()` → badge cabeçalho + widget + barra de XP
- `avaliarConquistas()` reavalia tudo (revoga/destrava) → `renderUI()`

8.3 Reset total

Configurações → `resetarDados()`

- `confirm()` → `storage.limparTudo()` → `localStorage.clear()` → `reload`

Fim da documentação. `background3d.js` (fundo animado em canvas) foi omitido por ser exclusivamente visual.